

Embedded Programming

Ranim Tom

September 12, 2025

Contents

1	Periph Driver Dev	5
1.1	Interrupt Programming Demo	5
1.1.1	Push Button Location	5
1.1.2	Peripheral interrupt interaction: EXTI engine	6
1.1.3	GPIO Interrupts	7
1.1.4	EXTI Register	8
1.2	Volatile Qualifier	9
2	GPIO Programming	11
2.1	GPIO General Concept	11
2.1.1	MCU Pin behind the seen	11
2.1.2	Modes for GPIO	13
2.1.3	Clocks	15
2.2	GPIO Registers	16
2.2.1	GPIO Ports, Pins, and registers	16
2.2.2	GPIO Mode Registers	17
2.2.3	GPIO Input and Pull-up Resistors	19
2.2.4	GPIO Output Mode	21
2.2.5	GPIO Alternate Function	22
2.2.6	GPIO Output type	22
2.2.7	GPIO Output speed register	22
2.3	Alternate Function	23
2.3.1	Rgisters side	23
2.4	Project Arch	24
2.4.1	Device Header	25
2.4.2	GPIO Driver	26
2.5	ToDo List Periph	27
2.6	Summary Periph drvier part 1	28

Chapter 1

Periph Driver Dev

1.1 Interrupt Programming Demo

In this section, we will see a practical application about interrupt programming, using the push button in the MCU.

1.1.1 Push Button Location

In [Figure 1.1](#), we see the diagram of a push button (this photo is from the user manual)

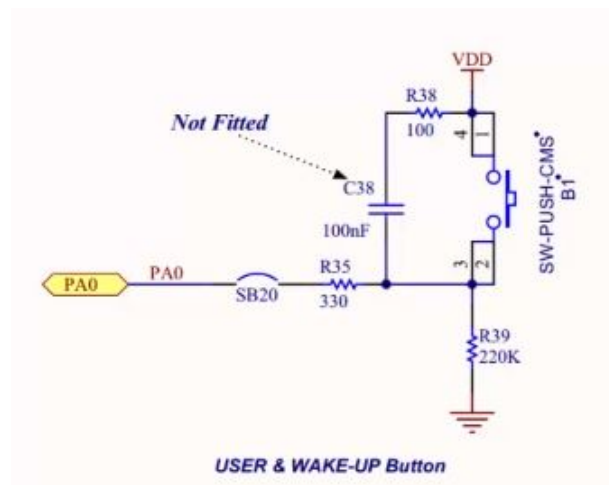


Figure 1.1: Push Button Diagram

- When we press the push button, VDD will be connected to point 2. And since we have R35 (the lowest resistance) compared to R39 (very high resistance), and R35 is from the pin A side, pin A will be pulled to high $\longleftrightarrow$ state of pin A is high when pressing the button
- When we release the button, pin A will be pulled to ground $\longleftrightarrow$ state is low when releasing the button

1.1.2 Peripheral interrupt interaction: EXTI engine

Before proceeding to programming, we need to understand how GPIO or any peripheral delivers their interrupt to the processor.

Note: this concept of peripheral interrupt processor interaction is *vendor specific*.

We know from chapter arm cortex, that when dealing with interrupts, we have peripheral side in the MCU, and the processor side. Since we are working with interrupts, we need to see how ***GPIO interrupts are passed to NVIC***.

To understand this concept, we open the *reference manual* at section 12.2: external interrupt/event controller (EXTI).

To handle peripheral interrupts, stm has what is called *EXTI*, that is external interrupt controller. It's block diagram is shown in [Figure 1.2](#).

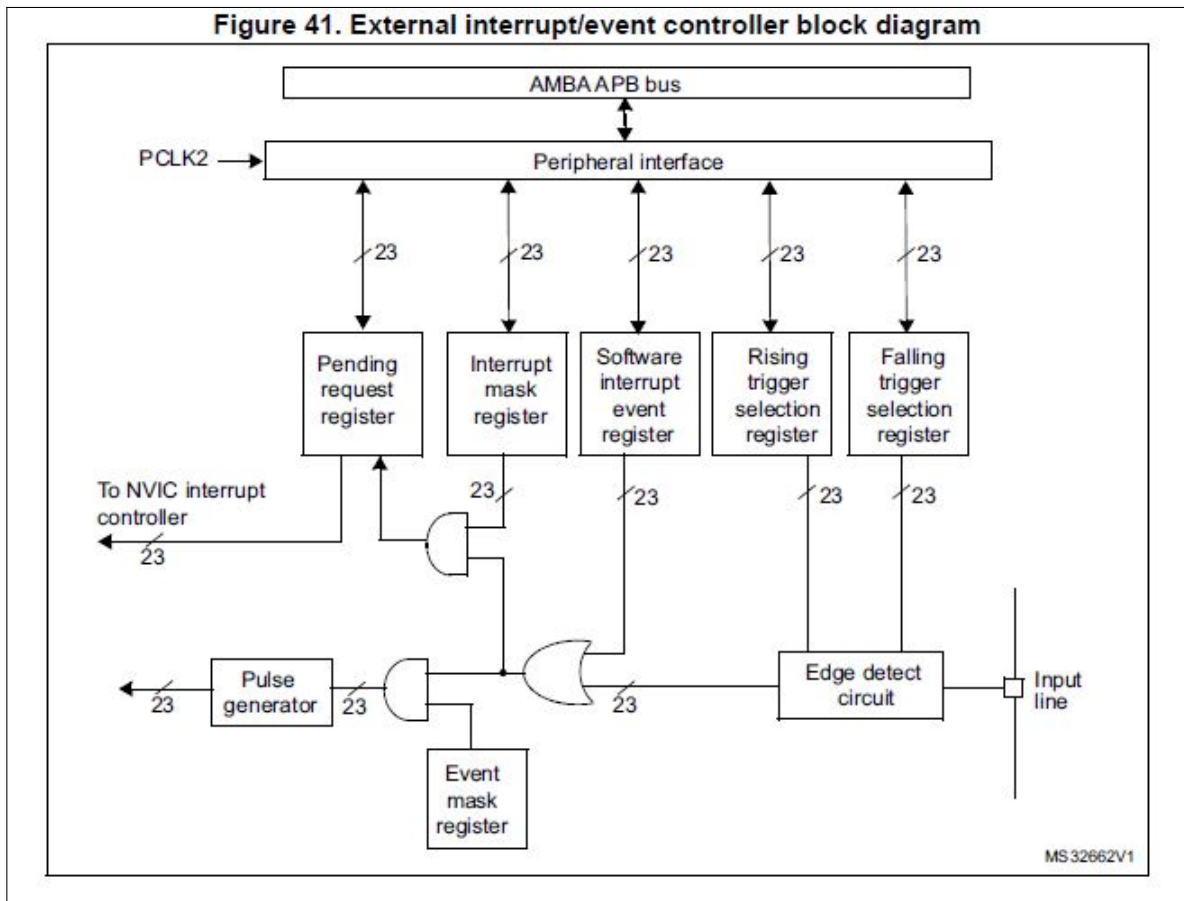


Figure 1.2: EXTI Peripheral from stm to handle some peripheral interrupts

We can see that this engine is connected to NVIC (which is a peripheral of the processor) using 23 connection. Which means, it captures 23 interrupt positions, or we can say it has 23 different IRQ numbers.

This means if we go to the vector table (table 61 in the reference manual), we should see 23 lines related to EXTI engine.

As example, we can see in IRQ 6 $\rightarrow$ 10, we have EXTI0 till EXTI4.

If we go also to IRQ 42, where we have the USB peripheral, we can see that the USB interrupt will go through EXTI, and not directly through NVIC.

1.1.3 GPIO Interrupts

As for GPIO, one thing we can notice that if we search the entire table 61 of interrupts, we don't found GPIO peripherals.

To understand the GPIO interrupts deliver, we go 12.2.5 in the reference manual, figure 42 (shown in [Figure 1.3](#))

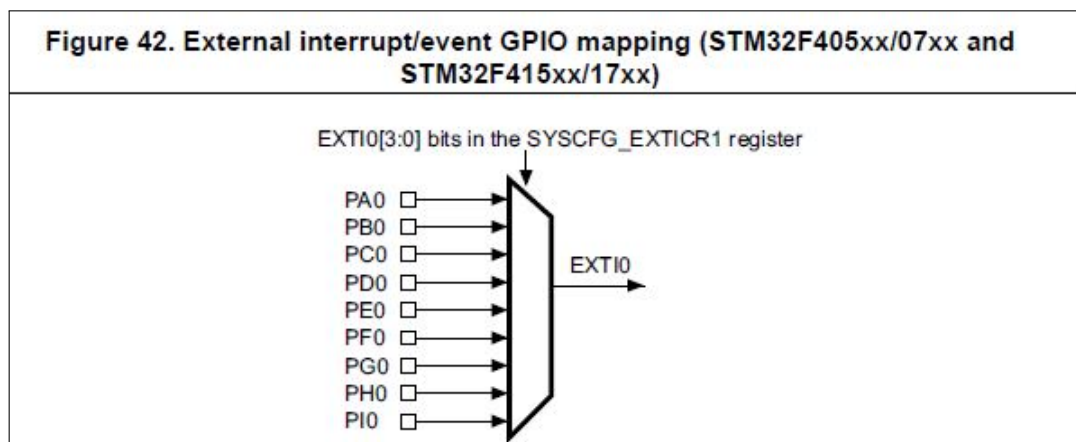


Figure 1.3: EXTI engine of stm to handle GPIO interrupts

- We can see that all 0th port (1st pin of each port) of the MCU deliver their interrupt through EXTI0 through a multiplexer, and by configuring the SYSCFG_EXTICR1 register.
- We see later that SYSCFG_EXTICR1 register select the link between a certain port and the EXTI0
 - for example, if we connect our button to PD0, we need to configure the link between PD0 line and EXTI0

Summary of Button Interrupt:

1. The button is connected to a GPIO pin of the MCU
2. Configure the GPIO pin to input mode
3. Link between GPIO pin and EXTI should established through SYSCFG_EXTICRx register
4. Configure the trigger detection (rising, falling or both) for relevant EXTI line

- In [Figure 1.2](#), we have several register such as Rising trigger selection, falling trigger selection

5. Implement the handler (C function) to serve the interrupt

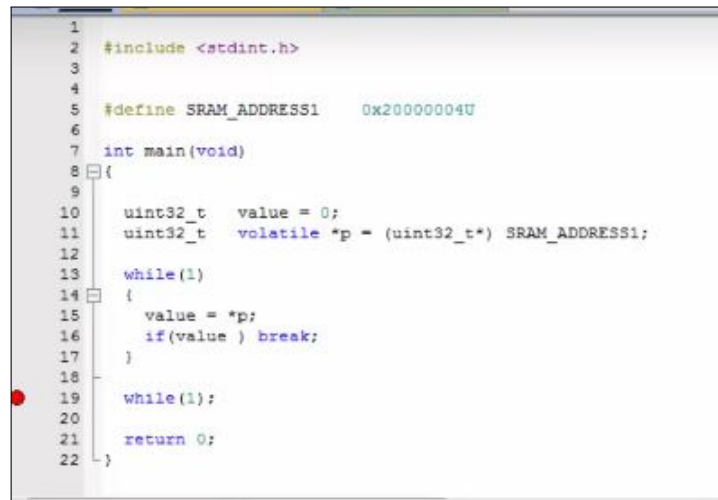
1.1.4 EXTI Register

EXTI register can be found in section 12.3 in the reference manual.

- EXTI_IMR (Interrupt mask register):
 - Since EXTI desing support for 23 line, only 23 bit of the EXTI_IMR can be manipulated
 - By default, EXTI doesn't poke the NVIC, we need to enable it through EXTI_IMR register, by setting some bit to 1

1.2 Volatile Qualifier

In [Figure 1.4](#), we have a simple application where:



```

1  #include <stdint.h>
2
3
4
5  #define SRAM_ADDRESS1  0x20000004U
6
7  int main(void)
8  {
9
10     uint32_t  value = 0;
11     uint32_t  volatile *p = (uint32_t*) SRAM_ADDRESS1;
12
13     while(1)
14     {
15         value = *p;
16         if(value ) break;
17     }
18
19     while(1);
20
21     return 0;
22 }

```

Figure 1.4: EXTI engine of stm

- we initialize some pointer variable to some address in memory
- we assign `value` variable by the value pointed out by the pointer
- If the value is not 0, we break the loop

The take away from this story is if we modify the optimization level of the compiler to O3 for example, even if we modify the value at runtime, the compiler will not read the value again because in his method (to optimize the code), he will skip reading value again and again because reading value needs several clock cycles, and he will try to save these clock cycles.

So even if we change the `*p` content, it will not take it, and the solution to that is to use the qualifier `volatile`

Volatile Qualifier:

- *To review later the concept of compiler optimization*
- *Role of volatile in C in general, watch some youtube videos about it*
- *Source is section 13 from the course*

Volatile Qualifier

Chapter 2

GPIO Programming

Now we start by GPIO programming part.

2.1 GPIO General Concept

- GPIOs are not only used for LED driving for example, but can also be used to:
 - Reading digital signals
 - Issuing interrupts
 - Generating triggers for external components
 - Waking up the processor

2.1.1 MCU Pin behind the seen

A GPIO pin is nothing more than 2 buffers with some enable line as shown in [Figure 2.1](#).

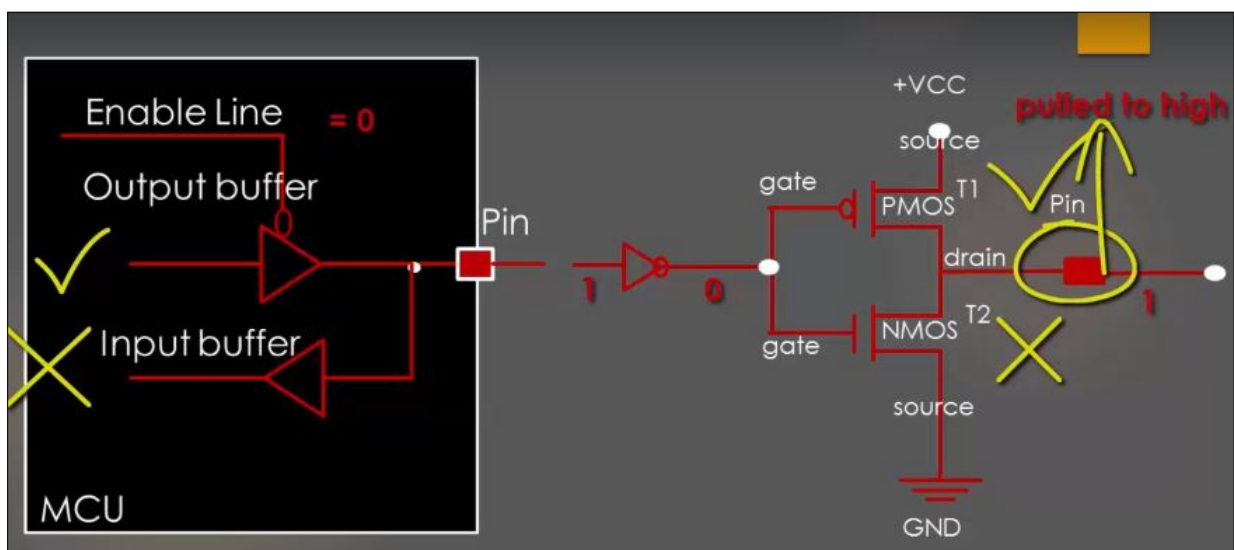


Figure 2.1: GPIO pin: output mode

- when enable line is 0, output buffer is get activated

- The buffer is nothing but a CMOS circuit, from PMOS and NMOS
- When writing 1 to the pin, T1 (transistor) is getting activated and T2 is deactivated, and the pin will be pulled to VCC and hence we obtain 1
 - When writing 0, T2 will be activated and T1 deactivated, hence the pin will be pulled to ground and we obtain 0 volt

Now we take a look for GPIO pin for input mode, show in [Figure 2.2](#)

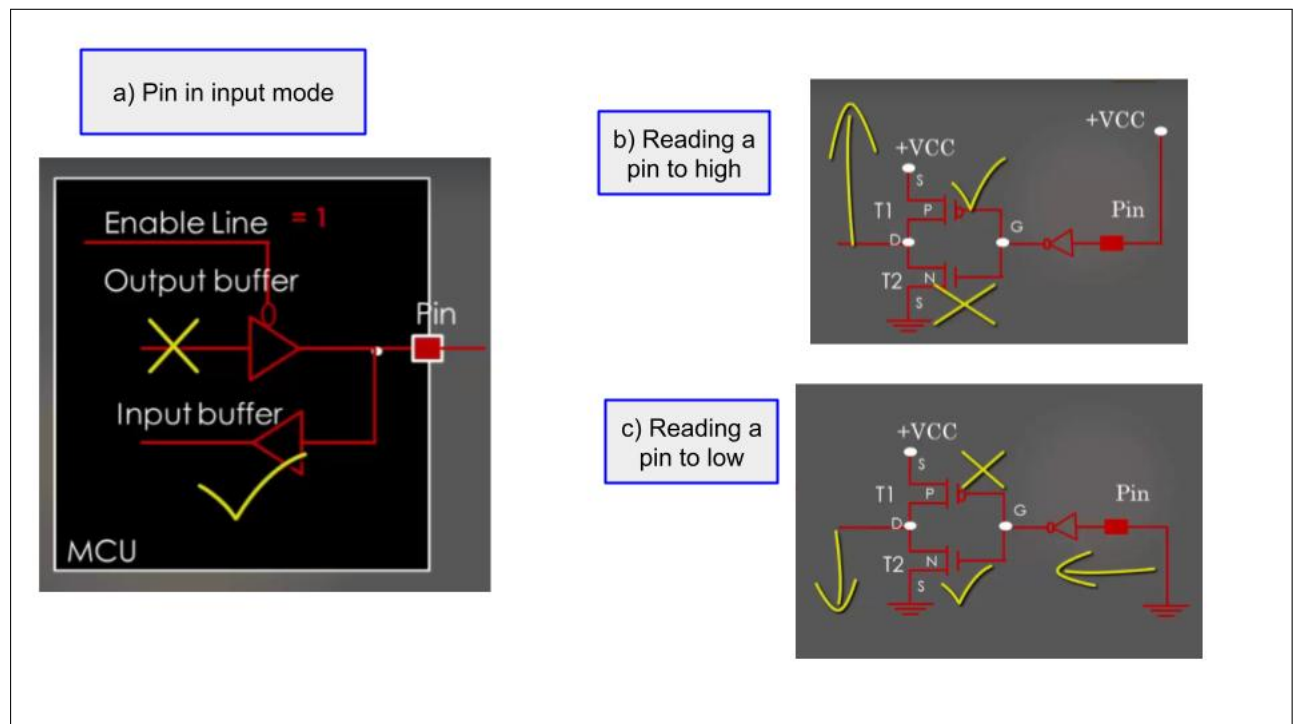


Figure 2.2: GPIO pin in input mode

- In input mode, the CMOS circuit behave as rotated circuit compared to [Figure 2.1](#)
- When pin is a high, T1 is enabled to high hence we read high
- Similar when pin is at ground, T2 is enabled hence we read low

The enable line (select input or output mode) is configured by control registers.

2.1.2 Modes for GPIO

High Impedance (floating) State:

Whenever we power up an MCU, the GPIO pins are, by default, in input mode with the *high impedance state* as shown in [Figure 2.3](#)

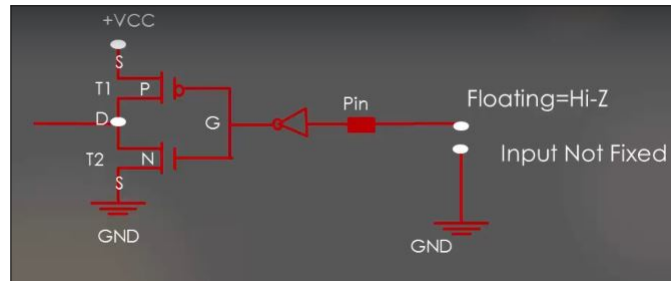


Figure 2.3: GPIO pin in high impedance state

- the high impedance state (also known as *floating state*) is keeping the pin neither connected to ground or high voltage
- a pin in a floating state may lead to leakage in current causing higher power consumption
- The reason for that is that in state, the pin will pick more noise current

Output Open Drain State:

[Figure 2.4](#) shows a pin in the open drain state.

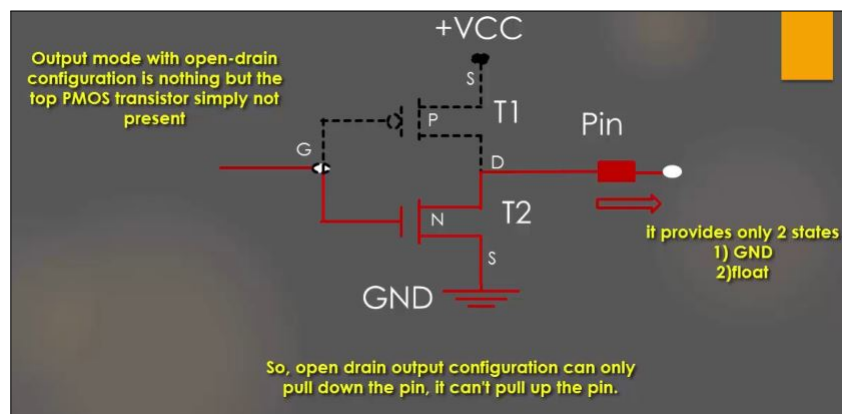


Figure 2.4: Pin in Open drain state

Open drain makes the pin useless (either ground or float state). For that, we need to use some internal pull-up resistor.

An example for LED driving is shown in [Figure 2.5](#).

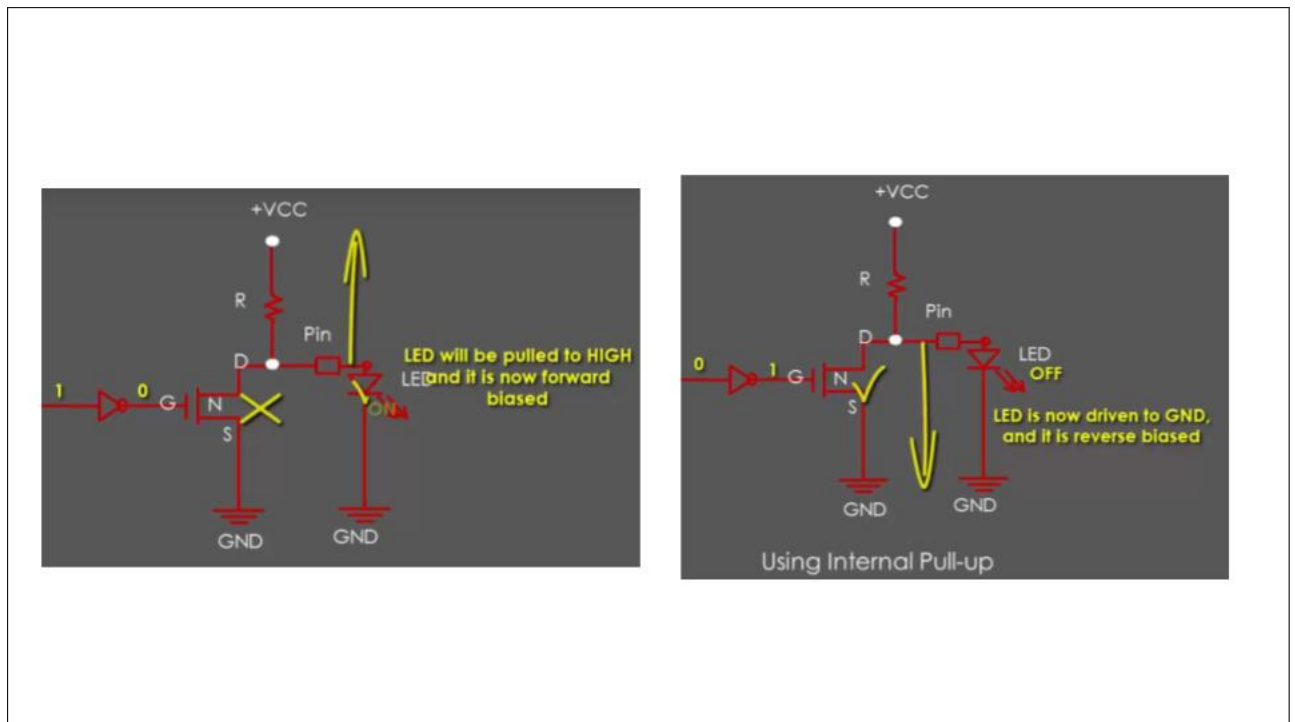


Figure 2.5: Open drain with LED driving application

- When 1 is input, the transistor takes 0 (because of the inverter logic), thus the transistor is canceled and the pin is pulled to high and the LED is turned on
- Internal resistors values are ranged from 10K to 50 kohm

Output Push Pull State:

Output push pull state is shown in [Figure 2.6](#)

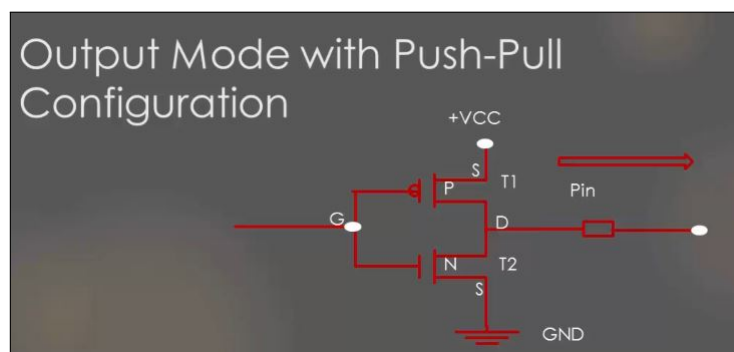


Figure 2.6: Output push pull state

- This state is called push pull because the output will be pulled between high or low depending on the 2 transistors, unlike the open drain state (see [Figure 2.4](#)) where the pin can be only pulled to ground, and we need some pull up resistors (as in [Figure 2.5](#)) to change the state high or low

- This lead to the 2nd difference: in push pull, we don't have internal resistors

Now as rules transistors for driving the output level:

- Top transistor PMOS will be on when the output is driven high
- Bottom transistor NMOS will be on when the output is driven low

push pull transistor rules:

push pull tr
sistor rules

- *To review that later, how the circuit (transistor) behaves when changing input voltage*
- *But to see when finishing GPIO programming if I need such a knowledge or not*

In [Figure 2.7](#), we have push pull configuration with LED driving application

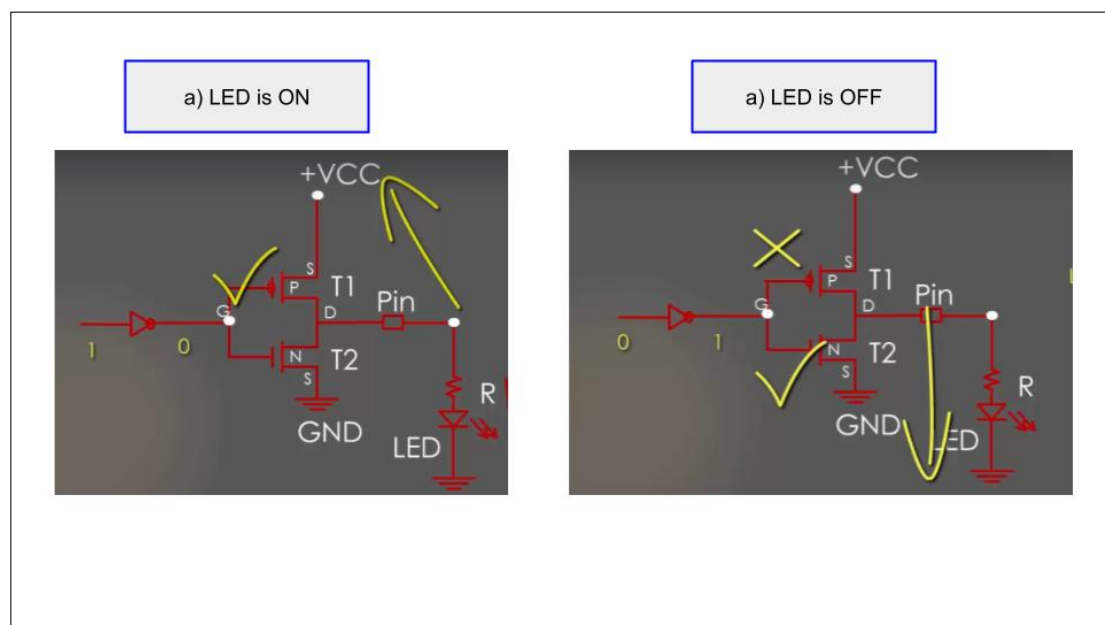


Figure 2.7: Output push pull state with LED

2.1.3 Clocks

Whenever we want to work with some peripheral such as the GPIO, we need to enable the clock.

In stm MCU, the RCC peripheral (Reset and Clock control) is responsible for enabling the clock for all other MCU peripherals, such as GPIO, timers, and also we for the processor side.

For GPIO, and since they are connected to AHB1 bus (see section 2.2 functional block diagram in the data sheet), the register responsible for this part is RCC AHB1 (see 7.3.5 in the reference manual)

2.2 GPIO Registers

After seeing GPIO concepts from circuit and electronic side, we now move to programming side.

2.2.1 GPIO Ports, Pins, and registers

In general in any MCU, each GPIO port is configured from a set of registers. Each register has a specific function, and all these registers communicate through a bus to the processor of the MCU. A generic block diagram is shown in [Figure 2.8](#)

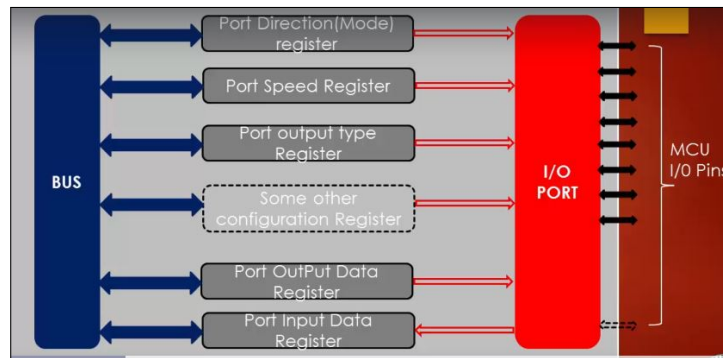


Figure 2.8: Output push pull state with LED

As a specific example for our MCU, the stm32f4- discovery board, in [Figure 2.9](#) is shown its block diagram (taken from data sheet section 2.2, Functional Overview)

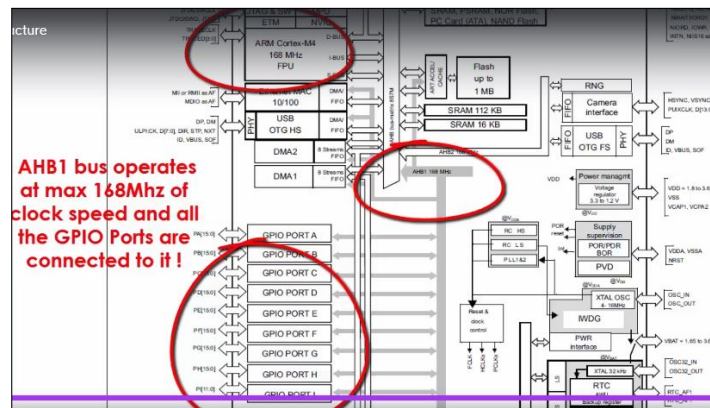


Figure 2.9: Output push pull state with LED

- In this MCU we have 9 ports A → I
 - Each port has 16 pin (from 0 → 15)
- Each GPIO port has a set of configuration registers as stated in [Figure 2.8](#)
 - The function and the configuration of each register can be found in the *reference manual*

- These ports communicate to the processor through AHB1 bus

One final note also to be stated (shown in [Figure 2.10](#)) about the ports

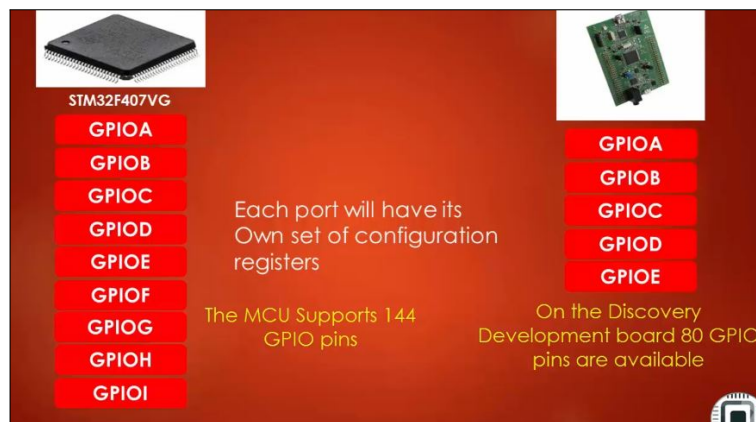


Figure 2.10: GPIO Ports on MCU and development board

- It is true that in the MCU we have 9 ports
- But the manufacturer of the development board has made 5 ports only available

2.2.2 GPIO Mode Registers

Now we start exploring different registers related to GPIO.

Recall that GPIO register or any register details are presented in the reference manual, and not the data sheet.

Also in the reference manual (at least with stm), each chapter is dedicated to some peripheral, and in the final section we see the details of registers.

Finally a helpful concept known as the register map, for example for GPIO, we can open the reference manual at section 8.4.11 and we have what we call the GPIO register map, and it contains the offset address with respect to the base address of the peripheral (this will be helpful in programming later).

Now let's start exploring different registers regarding GPIO peripheral.

GPIO MODER:

The **MODER** register is used to set a certain port to input, output or alternate function mode. Alternate function means we can configure the pin to do functions other than classical input or output, such as used for SPI signaling, transmission,...

There is also analog mode, used when dealing with ADC and DAC.

Each section of the register (each 2 bit) is used to configure 1 pin of a certain port, as shown in [Figure 2.11](#).

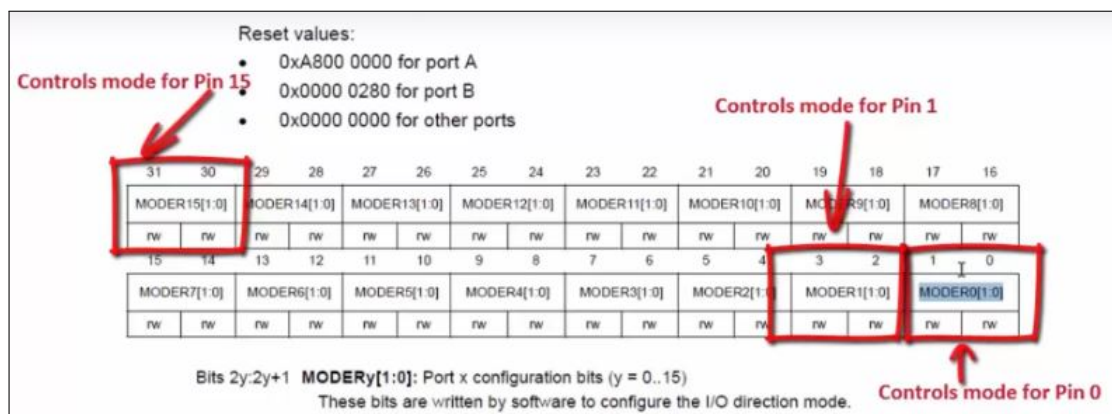


Figure 2.11: MODER configuration for pins of a port

Also we need to pay attention to reset values of this register for port A and port B, since they are not 0 when going under reset.

2.2.3 GPIO Input and Pull-up Resistors

In this part we explore the importance of pull up resistor in input mode.

First, we present electronic circuitry of a GPIO pin, shown in [Figure 2.12](#) (taken from the reference manual, section 8.3 figure 25)

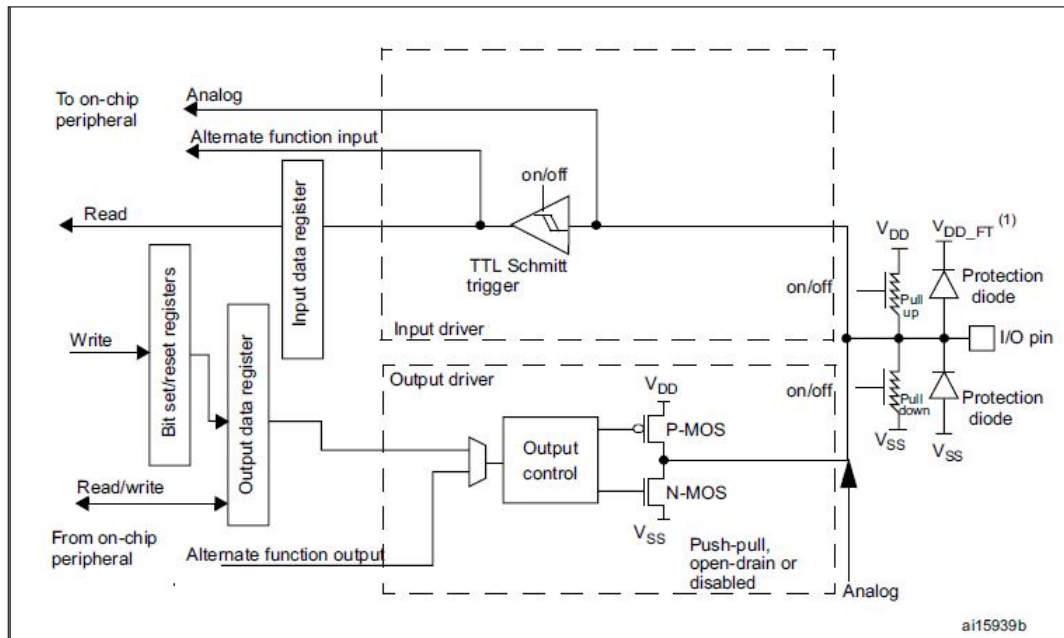


Figure 2.12: GPIO Pin electronic circuit

- When the input mode is activated for the pin, the bottom half of the circuit (output driver) will be canceled
- Recall that by default the pull up resistors are deactivated

As a result, when selecting input mode for the pin, we obtain the following diagram shown in [Figure 2.13](#)

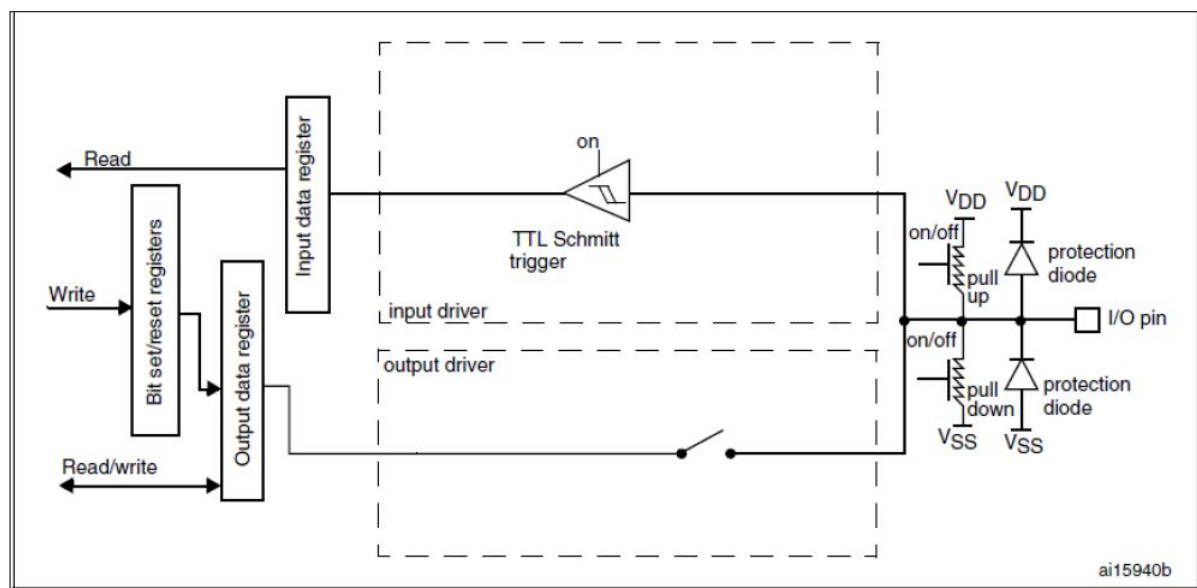


Figure 2.13: GPIO Pin electronic circuit

The input pin now is at floating state, and this state can cause voltage fluctuation.

As application, let's imagine we have a switch connected to the pin, and its grounded from the other side, and we need to read from the switch.

If the switch is closed, the IO pin will be connected to the ground (the whole path is grounded), and we always read 0 from the input data register **every AHB1 clock cycle**, that is because GPIO port are connected to AHB1 bus as seen in [2.2.1](#).

But when the switch is open, the IO pin is neither grounded neither in high state, it will change all the time at the input data register (0101), that's why we need to activate the pull up resistor in input mode when reading from a switch.

Because if we have some code saying if input-state == high, then turn the motor on, it will result in a bug since the values at the input data register is keeping of changing.

ut switch
ding

Input switch reading:

- I didn't completely understand how when switch is on (open), the pin state will keep changing
- To see this later when reviewing electronics and circuit concept of the flow of current, voltage state,...
- Source: see video 58 section 15 from perih driver 1 course

Take a way: when working with IO pin in input mode, we activate pull up resistor to have proper state of the pin, high or low.

All modern MCU provide directly internal pull up resistors, and if not (to verify from data sheet and reference manual), we need to provide external pull up resistors.

2.2.4 GPIO Output Mode

In output mode, there is 2 way: push pull and open drain.

Push Pull:

The flow is shown in

hand note open drain:

hand note
open drain

- To insert later the diagram in hand notes, GPIO programming file
- Still need to review the concept of transistors, when the transistor will be opened and not depending we have 0 or 1

Open Drain:

Recall from 2.1.2, that open drain we have only N-MOS transistor, and the pin state is either float or GND (see Figure 2.4).

Open drain diagram:

Open drain
diagram

- To insert later the diagram in case of open drain from hand notes, GPIO file

When reaching a float state for IO pin in case we have 1 as input, the pin will be at float state (see diagram in my hand notes), so we need to use pull up or pull down resistors to have a correct state in case of float IO pin.

LED Driven Application for Open drain configuration:

In case we connect a LED for the IO pin, and we input 1, the current will pass through the LED because we are using internal pull resistor. However, since the internal pull value is high (in the range of 40 Kohm, see the data sheet for values of the MCU), the current will be weak and the LED will not turn on.

That's why we need to use *external pull up resistor* of values let's say 1 kohm. By this way, we ensure no floating state at IO pin, and strong current will go through LED because we decrease the resistance value.

Using which mode:

It really depends on our application. If we want our circuit to have 2 states high and low, and this circuit drive some other mechanism like a relay, motor, ... , we can always use push pull configuration.

In some application, the IO pin is required to be in floating state, so we need to use open drain, as in I2C specification, where it is required that the pin to be in open drain configuration.

Note: we should mention that in output mode, and unlike the input mode, the input circuitry of the IO pin (see Figure 2.12) will still ON, and will not be deactivated as in input mode.

This mean we can read the state of the IO pin using the input data register.

Input and Output:

Input and
Output

- Does this mean that a pin can be configured as input and output at the same time ?
- Source: video 61 section 15 of periph driver 1

2.2.5 GPIO Alternate Function

Alternate function pins are in general used by *some other peripheral of the MCU*, such as for example UART Tx or UART Rx, timer input channel, adc input channel, or some data line of the I2C peripheral, some clock signaling, ...

The alternate function will be explored later once we start with more advanced topics, such as communication interfaces as I2C, SPI, ...

Now getting back to [Figure 2.12](#), we see that:

- Alternate function can be also in 2 mode: input and output
- For alternate function output: the data now will come from the alternate function line, and ***not from output data register***.

In other words, the ***output data register will not affect the state of the IO pin***

Example of usage: UART Tx side.

- For alternate function input side: whatever the input we read, it goes to alternate function input line.

In case on input, and unlike output side, it get updated to input data register

Example of usage: UART Rx side, CAN Rx side, ...

2.2.6 GPIO Output type

Whenever GPIO pin is in output mode, this register is used to choose the type of the output pin, that is push pull or open drain.

It can be viewed in reference manual in section 8.4.2

2.2.7 GPIO Output speed register

This register is used to configure the speed (in MHz) of the transition from L $\rightarrow$ H or H $\rightarrow$ L. This register can only be used ***if the pin is in output mode***.

In the reference manual (section 8.4.3), we have several option for speed: 00 for low speed, 01 medium speed, ... The exact values for these speed can be found in the data sheet (table 50 page 117).

For example, for 00, we have 4 different speed in MHz. These speeds depends on conditions, that the capacitance and the voltage source.

Output speed
register

For our application for now, we can ignore this register and let at 00.

Output speed register:

- To see later some use cases, like implementing the I2C protocol if our MCU doesn't support this protocol, and we need to support it in our MCU by using bit banding technique
- Source: video 64 from section 16 in periph driver course 1

2.3 Alternate Function

As seen earlier, a GPIO pin can have 4 different functions: input, output, alternate function, and analog.

We explain here the alternate function concept.

A gpio pin in alternate function mode, can have 16 possibilities. To take a concrete example, let's see what is the possible alternate function for PA8. For that, we need to open *data sheet*, at table 9 in chapter 3.

The column side at the left contains the port, and the header of the table contains the different functions available, such as input for timer, SPI,...

For PA8, we see that:

- AF0 is MCO1 (microcontroller clock output 1)
- AF1 is TIM-ch1: channel 1 for the timer 1 peripheral
- AF2 is nothings, this means it behaves a normal gpio pin
- And so on for different blocks

So in summary, for PA8, we have 6 possible alternate function.

For each pin in each port, we need to refer to this table to choose the write one depending on our application.

2.3.1 Registers side

To configure an alternate function register, we have 2 register in the reference manual: GPIO alternate function *low register*, which configures from pin 0 (ALFR0) → pin 7 (AFRL7), and the GPIO alternate function *high register* which configure the higher pins from 8 → 16

2.4 Project Arch

Now we start with some coding to apply the theory. [Figure 2.14](#) shows the roadmap we will do in this part.

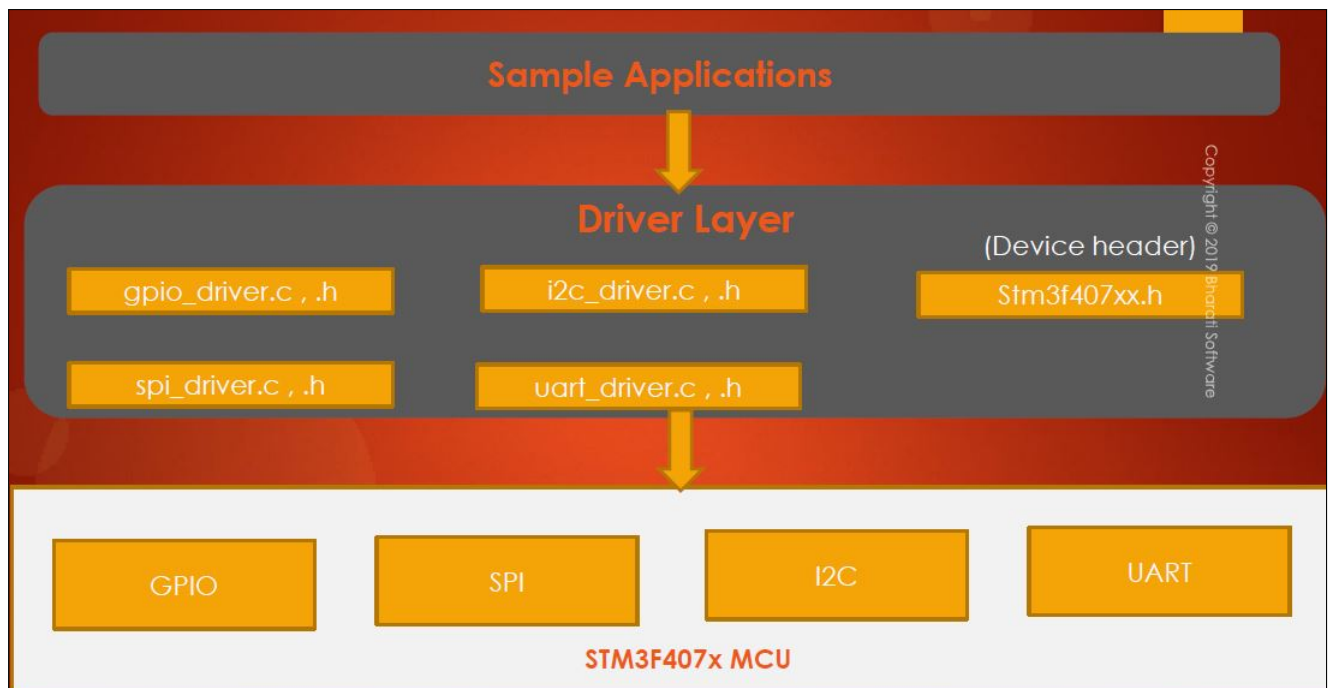


Figure 2.14: Peripheral 1 project map

In the next section we start implementing what is called device header, which contains some information needed for driver layer files, such as base address for some peripheral, base address for memories, All the details are show in [Figure 2.15](#).

What is Device Header file and what it contains ?

This is a header file ('C' header file in our case) which contains Microcontroller specific details such as

- 1) The base addresses of various memories present in the microcontroller such as (Flash, SRAM1, SRAM2, ROM, etc)
- 2) The base addresses of various bus domains such as (AHBx domain, APBx domain)
- 3) Base addresses of various peripherals present in different bus domains of the microcontroller
- 4) Clock management macros (i.e clock enable and clock disable macros)
- 5) IRQ definitions
- 6) Peripheral Register definition structures
- 7) Peripheral register bit definitions
- 8) Other useful microcontroller configuration macros

Copyright © 2019 Bhoot Software

Figure 2.15: Device Header content

2.4.1 Device Header

As 1st step we start by device header.

- We can download from stm website the device header spec for stm407 discovery board, but for the sake of practicing reading and coding , we will implement it by scratch
- From the reference manual, we search for the addresses of the memories
- Then the buses: APB1 since it starts the first
- Important Concept: The peripheral are hanging on buses, so we code address of peripherals as starting address of a certain bus plus some offset
 - the offset can be found for each peripheral in what is called the **register map** in the reference manual
- Once last note: once the header file for the specific board is finished, don't forget to include its path in stmcube IDE
 - build setting in eclipse/stmcube: To see later how to build using custome build settings like make or cmake
 - compare the 2 approach, that is using the custom build or the build system of the IDE

build setting
in eclipse/stm
cube

2.4.2 GPIO Driver

Now we move to the 2nd step, the gpio driver layer.

Now, the function of the driver layer is to do the configuration provided by the user application. In other words, the user will ask to change some gpio registers, which means the user file contains the specification, the driver takes these specification and apply the configuration to the register according to the user specification.

2.5 ToDo List Periph

- Maybe to review later some electronics concepts, such as pull up and pull down resistor, how the current is flowing when changing input (0 or 1)
 - I saw these concepts in [section 2.1](#)
- To redo later video 54 from section 14 of the course, when finishing reviewing electronics concept

2.6 Summary Periph drvier part 1

- Interrupt concepts
 - Interrupt programming contains 2 side: the peripheral side and the processor side
 - Not all interrupts go directly to NVIC, some of them yes (like SPI for example), and some of them no (through EXTI engine)